pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

Hierarchical Clustering and Cluster Profiling

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

From the top down to the bottom up

Yesterday: K-means

Choose k first. Four centres move until every one of 821 bonds sits with its nearest centre. Four labels come back.

Today: hierarchical, on issuers

150 issuers, each a cluster of one. Join the two closest, repeat until one is left. Every merge is kept: choose k by cutting the tree.

Then the profile: counts and medians in bp, years, and billions, the part a desk reads.

Invented issuers and generated spreads: a cluster is a group of similar rows, never a ranking.

How agglomerative clustering works

Start	Every issuer is a cluster of one
Join	The two closest clusters are joined, and the join is recorded with its distance
Repeat	Until one cluster is left: n issuers give n - 1 joins
Nothing chosen	No k, no starting centres, no seed
What comes out	The full list of joins, cut into clusters afterwards at any height

One row per issuer, and a distance by hand

Column	From the bonds
median_log_oas	median of log OAS
median_duration	median, years
bonds	how many bonds
amount_bn	sum outstanding, USD bn
score	issuers.csv rating, 1 to 18

```
distance = sqrt(sum of
  (difference in each scaled column)^2)
```

2.0227

BEZE to BEZH

0.2075

closest: CLAY and WYXR

11,175 pairs of issuers. The closest pair is the first merge.

Scaled first: a difference of 1 is one standard deviation in any column.

Merging step by step

Merge	Joins	With	Height
0	CLAY	WYXR	0.2075
1	ELVX	GALY	0.2586
2	BEZN	VYRN	0.3289
3	THAX	XANH	0.3628
4	HESA	KEZA	0.4167
5	CLAL	YARN	0.4331
6	FESR	NEVN	0.4447
7	CLAX	ELVN	0.4491

How scipy numbers clusters

0 to 149: single issuers. The cluster made by merge i is 150 + i. Two single issuers join at their distance.

Merge 42: the first with a formed cluster

RULE joins cluster 169 at 0.7167, making 3. Every merge before it pairs two single issuers.

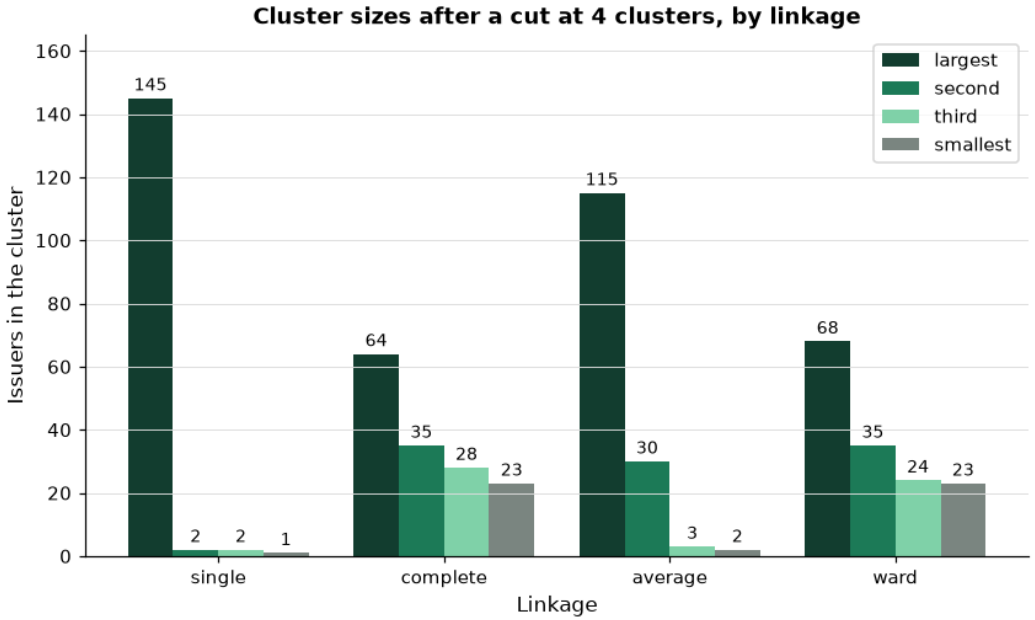
How the four linkages work

The need	Between two clusters, distance needs a rule: the linkage
Single	The closest pair: it can chain clusters together through one near pair
Complete, average	Complete: the farthest pair, compact clusters. Average: the mean over all pairs
Ward	How much the merge increases squared distances to the centre: compact clusters of similar size
What comes out	A different tree for each linkage; ward is the course's default

Linkages: four meanings of close

Linkage	Distance between two clusters
single	the closest pair
complete	the farthest pair
average	the average over all pairs
ward	the rise in squared distance to the centre

Single linkage chains: 145 of 150 issuers in one cluster. Ward's cut: 68, 35, 24, 23.



Same distances, four rules, four trees. The course uses ward, Euclidean, on scaled columns.

Four trees, one cut each

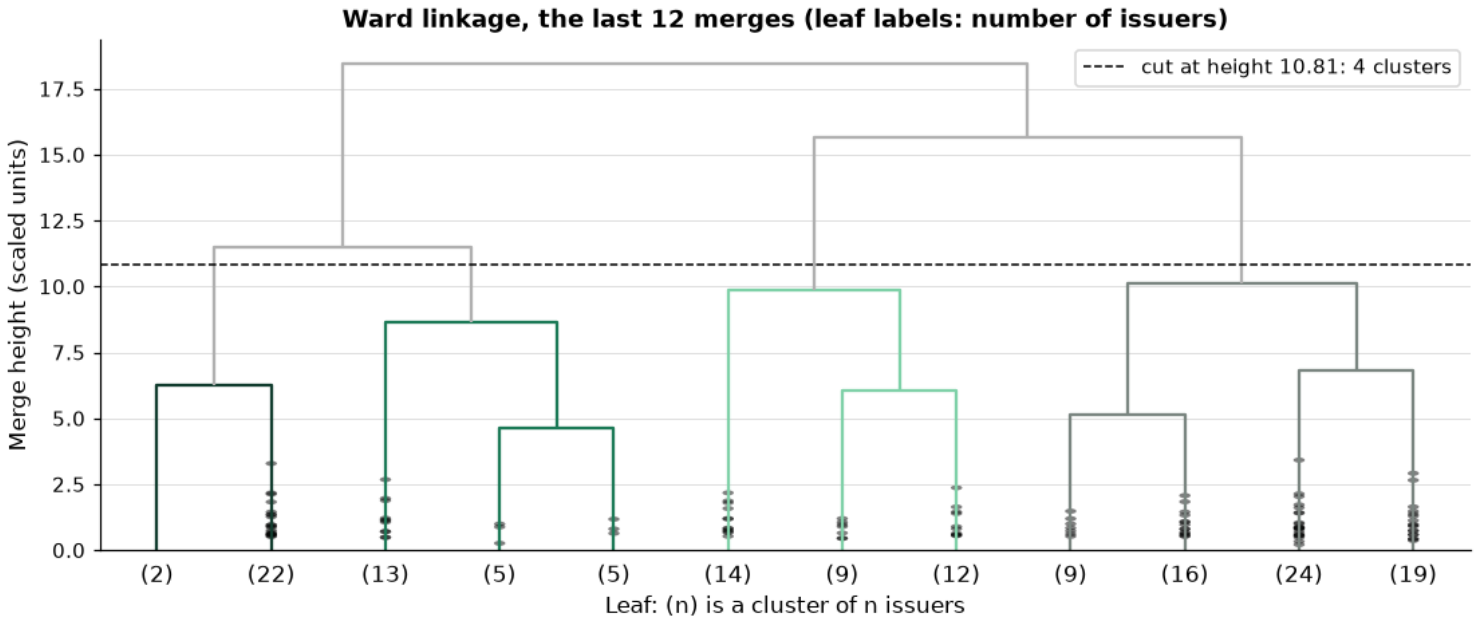


Single's cut sits low and slices off stragglers; ward's sits high and divides the issuers into groups of similar size. Heights are not comparable across panels.

How a dendrogram works

The tree	The issuers along the bottom; each join is a bracket at the height of its merge
Heights	Low brackets join close issuers; high ones join clusters far apart
A cut	A horizontal line across the tree: the vertical lines it crosses are the clusters
A long stretch	No joins over a wide range of heights: clusters that stay apart
You control	The linkage, and where to cut

The dendrogram, and where to cut

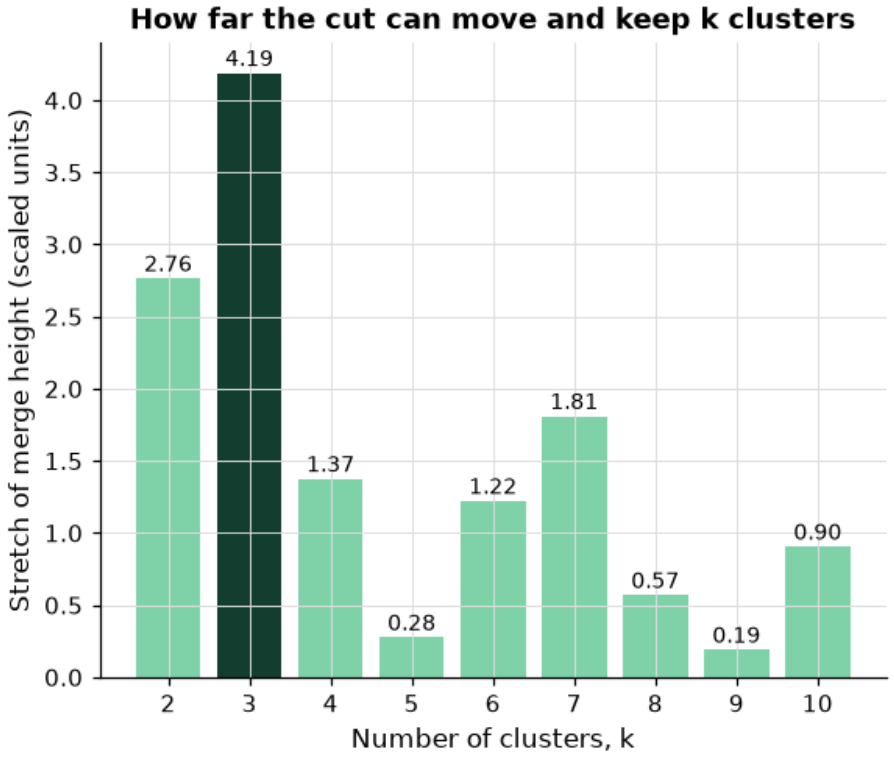


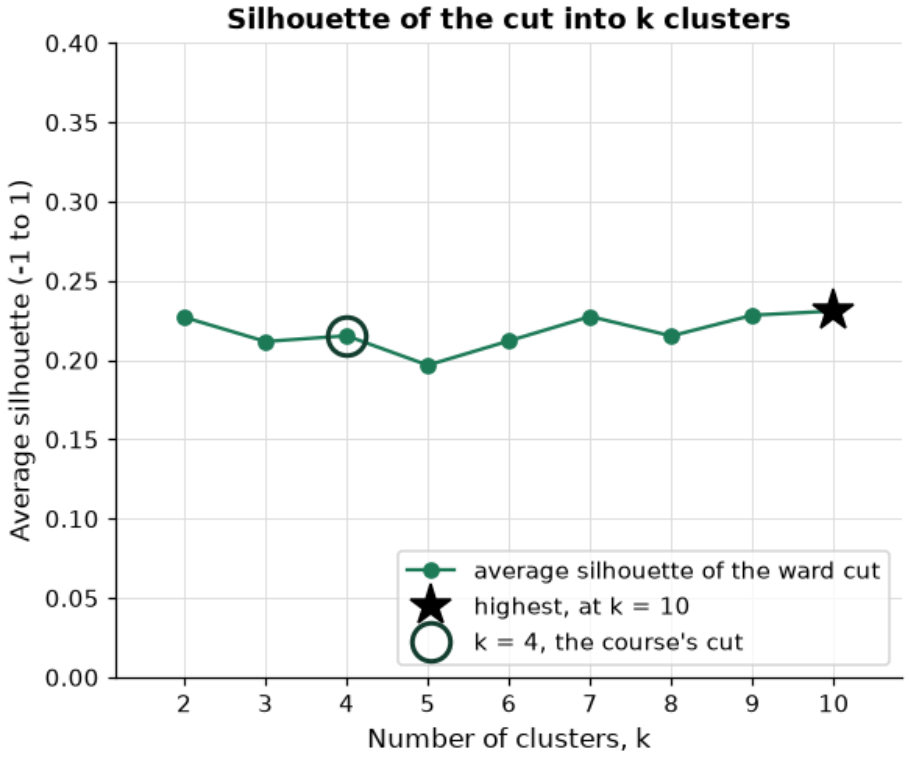
Height	Cut below: k
18.4445	2
15.6804	3
11.4930	4
10.1186	5

Cut at 10.81: 4 clusters

A judgement. The widest gap says 3; the silhouette is flat across k. Four keeps day 12's k and splits by duration.

How far the cut can move, beside the silhouette





The stretch is longest for k = 3 (4.19); the silhouette is nearly flat (0.1968 to 0.2310). Four is a judgement.

How cophenetic correlation works

The problem	A tree replaces all the pairwise distances with a short list of merge heights
Cophenetic	The height at which the tree first puts two issuers in the same cluster
The correlation	Between those heights and the original distances, over every pair of issuers
The reading	Near 1: the tree keeps the distances faithfully. Lower: it bends them
What comes out	One number for each tree, to compare linkages on one scale

Cophenetic correlation

Cophenetic distance: the height at which the tree first joins two issuers. Its correlation with the real distances, over every pair, says how faithfully the tree keeps them.

Linkage	Cophenetic
average	0.6317
ward	0.5212
complete	0.4903
single	0.4637

Two pairs

CLAY and WYXR: distance 0.2075, cophenetic 0.2075. The first merge.

BEZE and BEZH: distance 2.0227, cophenetic 15.6804. Different branches.

Average keeps distances best, yet its cut gives 115, 30, 3, 2. Look at the cut too.

How fcluster cuts a tree

The cut	Every issuer below the cut line joins the cluster its bracket belongs to
maxclust	t = 4: the highest cut that leaves at most 4 clusters; the course's convention
distance	Cut at a height that you choose
What comes out	One integer for each issuer
Numbering	By size, largest first, so the numbers mean the same on every run

Cutting the tree with fcluster

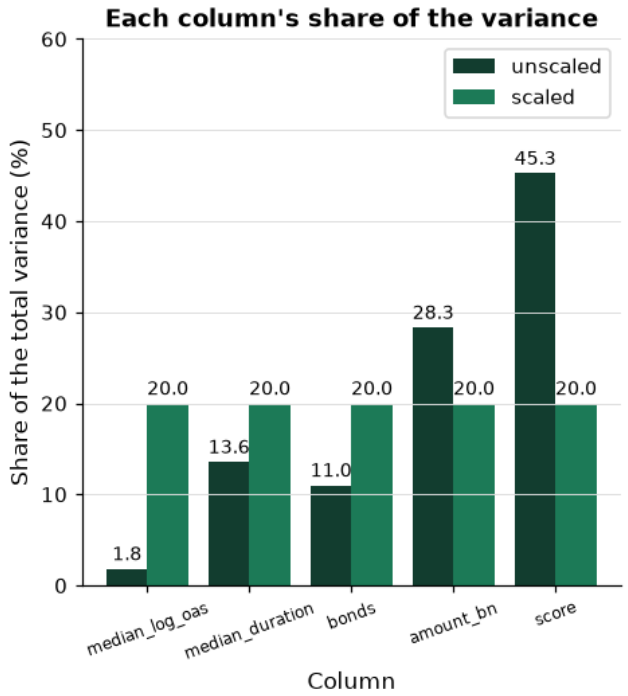
```
fcluster(merges, t=4, criterion="maxclust")        # the course's convention
fcluster(merges, t=cut_height, criterion="distance")  # the same clusters here
```

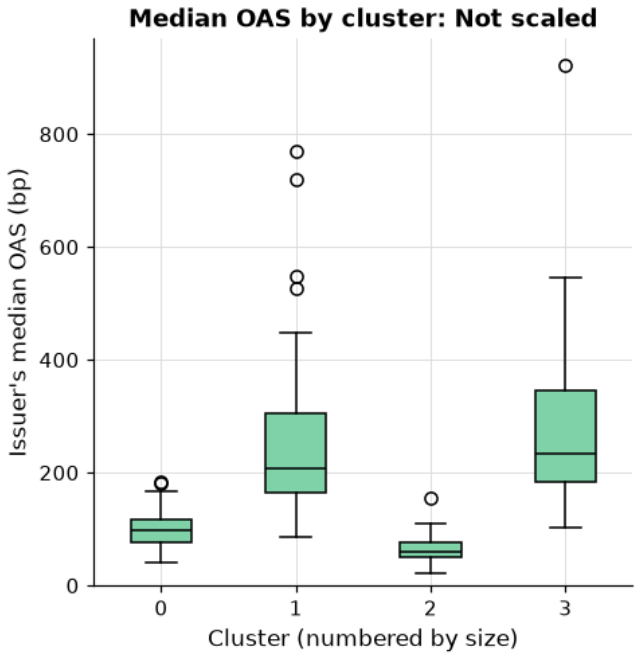
68	35	24	23
cluster 0	cluster 1	cluster 2	cluster 3

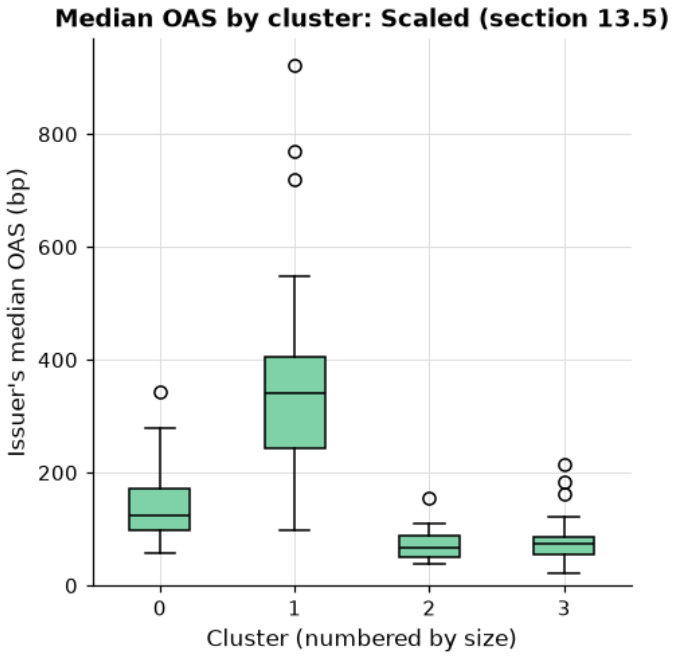
Numbered by size, largest first; a tie goes to the cluster holding the earlier row. AgglomerativeClustering, ward, 4: adjusted Rand index 1.0000, the same groups.

Cluster 0 is first in size, and in nothing else.

Scale first, on a tree too







Unscaled, the score carries 45.3% of the variance and the log spread 1.8.%. The unscaled clusters' median OAS spans 60 to 232 bp; the scaled, 68 to 342.

How a cluster profile works

The problem	A cluster number says who belongs together, not what they are like
The profile	For each cluster, its row count and the median of every column
Own units	From the original table, in bp, years, and billions, not the scaled columns
Why median	One wide issuer does not pull it
What comes out	One row per cluster, read as a description and never as a ranking

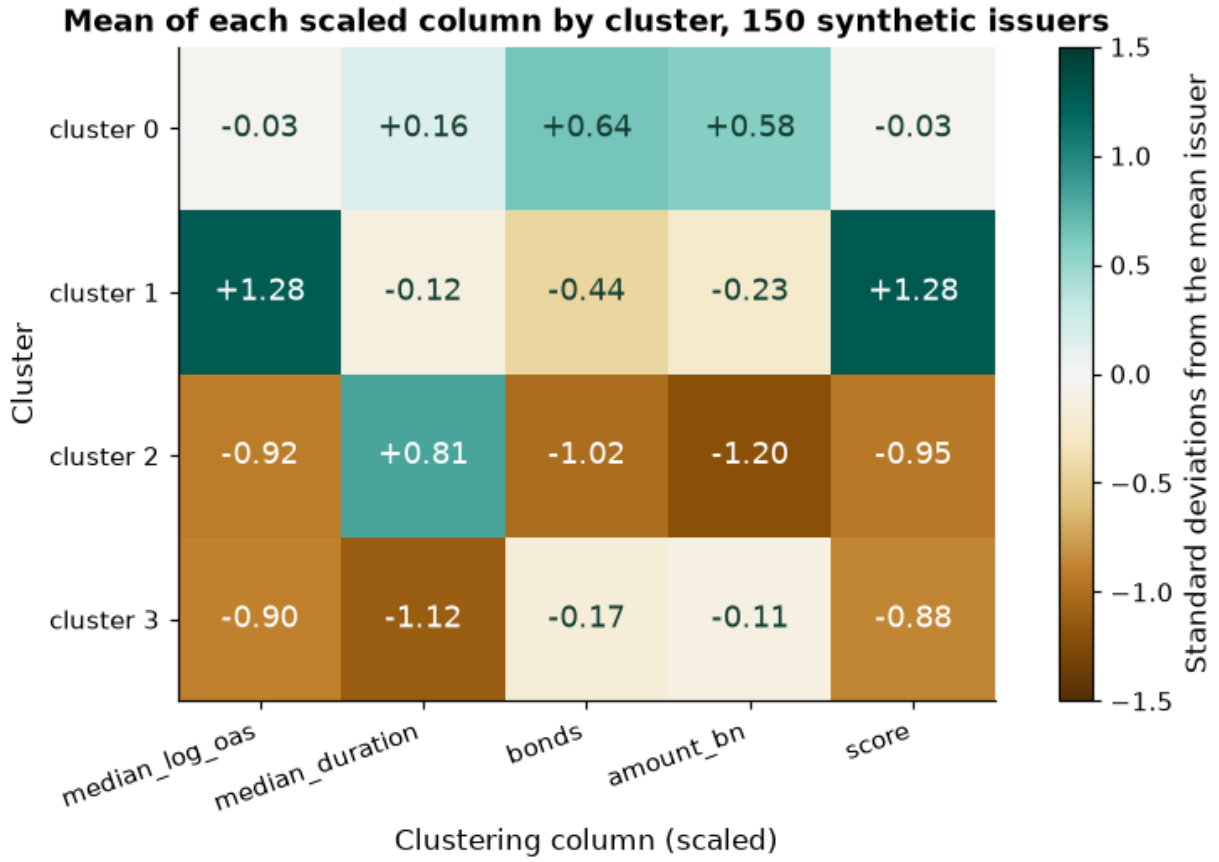
The profile, in the data's own units

Cluster	Issuers	OAS (bp)	Duration (yr)	Bonds	Amount ($bn)	Score
0	68	124.5	6.48	7.0	8.70	9.0
1	35	342.0	5.23	4.0	6.05	14.0
2	24	68.0	7.32	3.5	4.03	5.0
3	23	73.0	3.57	5.0	7.50	6.0

cluster_profile: count and medians of the unscaled rows. 53 tests pass. Clusters 2 and 3: similar spreads, split by duration.

A profile describes a group. It never ranks one.

Which columns made each cluster

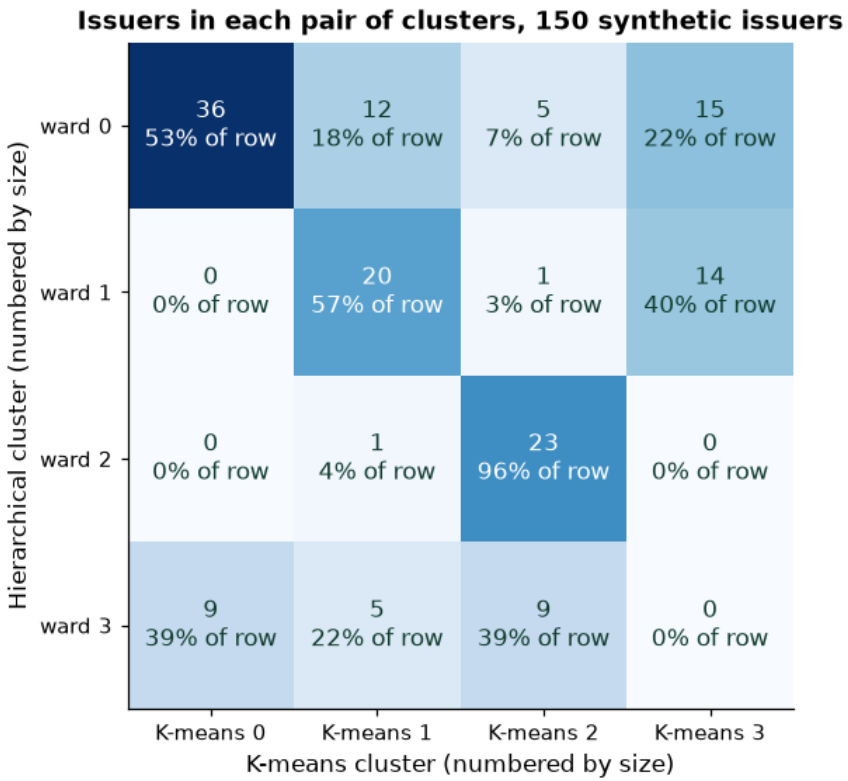


Read each row

Cluster 1: the two credit columns. Cluster 0: the two size columns. Clusters 2 and 3: low spreads, split by duration.

Scaled units say which columns; the profile says how much.

K-means against hierarchical, same issuers



0.2552

adjusted Rand index

0.2153

silhouette, ward

0.2577

silhouette, K-means

Read across: ward's cluster 2 lands almost whole in one K-means cluster (23 of 24); ward's wide-spread cluster 1 is split by size.

Neither is the true one: two rules impose four groups.

What a profile may say

Allowed names

cluster, cluster 2

a group of similar rows

counts and medians in units

Never called

good, bad, safe, risky

cheap, rich, mispriced

a ranking, a signal, a trade

Median log OAS and score: correlation 0.97, credit counted twice. Bonds and amount: 0.82, both random draws in the generator.

A group is only as meaningful as the columns that built it.

Excel side by side

Excel side by side (tested in Excel)

`=STANDARDIZE(B2,AVERAGE(B$2:B$151),STDEV.P(B$2:B$151))`	equal to StandardScaler
`=SQRT(SUMXMY2(J2:N2,J3:N3))`	2.0227, BEZE to BEZH
`=SQRT(SUMXMY2(J18:N18,J137:N137))`	0.2075, CLAY to WYXR
`=COUNTIF($H$2:$H$151,0)`	sizes 68, 35, 24, 23
`Pivot table: Count, Average`	no median in its list
`=MEDIAN(IF($H$2:$H$151=0,G$2:G$151))`	median OAS 124.5 bp, exact

Excel has no hierarchical clustering: the tree is built in Python, the numbers around it in Excel.

Save the day: the size of a commit

1. Module

mlkit.py, test_mlkit.py, .gitignore

2. Results

results.csv and profile.csv, 4 decimals

3. Stat

git diff HEAD~1 --stat

```
profile.csv | 5 +++++
results.csv | 2 ++
2 files changed, 7 insertions(+)
```

HEAD~1 is the commit before the last; --stat is one line per file. Read it before a long diff.

The AI check

The docstring is the prompt. The code that comes back runs, and has one modelling bug.

1

Docstring

Which rows fit, which rows report, and the units: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the profile is computed on the scaled columns and labelled median OAS in bp: 1.3502 for a cluster at 342.0 bp.

Code that runs is not the same as code that is right.

What you can do now

- Start with every issuer alone; merge the closest, 149 times
- The linkage decides what close means for a group
- Cut where you judge best, with reasons; number by size
- Cophenetic correlation: the tree, not the cut
- Profile in bp, years, billions; never rank a cluster

Week 3, Day 4 PCA and t-SNE: summarise the curve's columns, picture the bonds